Adam Campbell

Tyler Gill

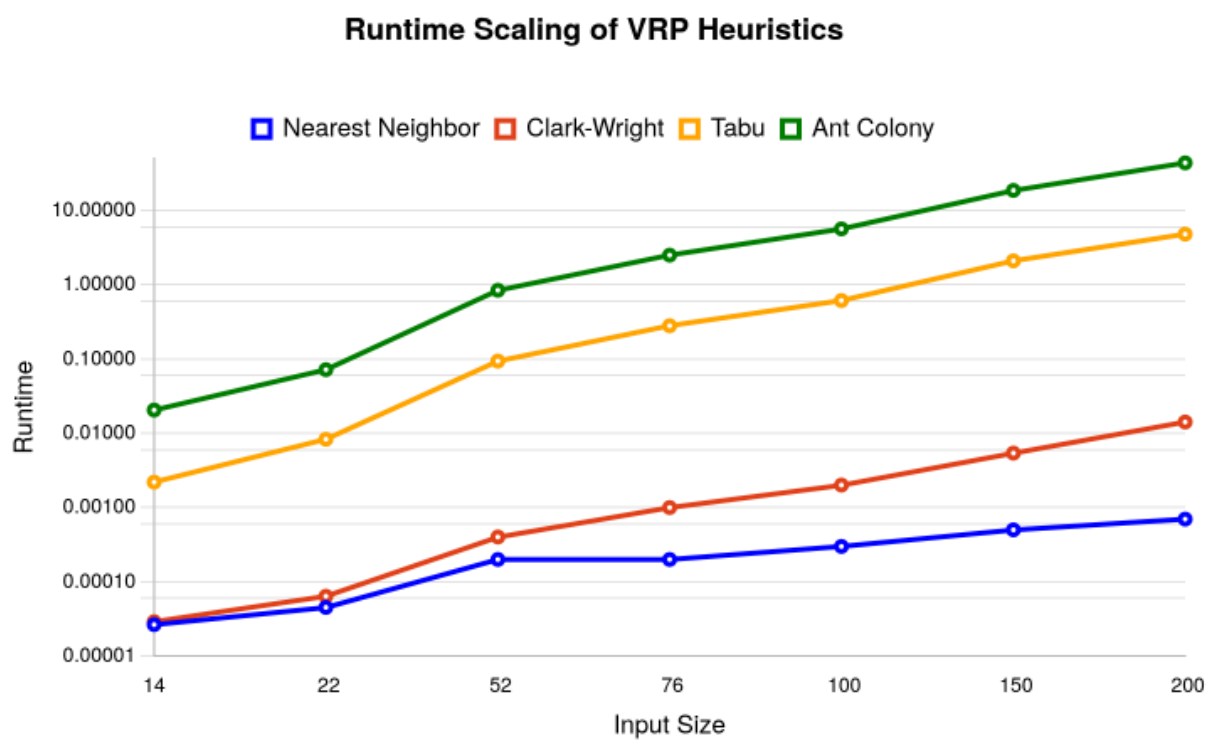
Tristan Worthington

***Runtime Complexity and Scaling Behavior of VRP Heuristic Methods***

The problem we chose to research for this project was the Vehicle Routing Problem (VRP). The VRP is dedicated to finding the most efficient route between a set of points for one or more vehicles to service a collection of customers that are met with constraints like capacity and distance. Given that the VRP contains the Traveling Salesman Problem (TSP) as a special case, it can be classified as NP-Hard, and as such, exact solutions will become more and more computationally unrealistic as the number of customers rises. Along with the nature inspired algorithm, the Ant Colony Optimization, the primary methods used in examination of the VRP were the Clarke-Wright Savings algorithm which served as the benchmark, the Nearest Neighbor Heuristic, and Tabu Search, each of which were chosen due to the fact that they are common solutions to the TSP which is the basis for the VRP.

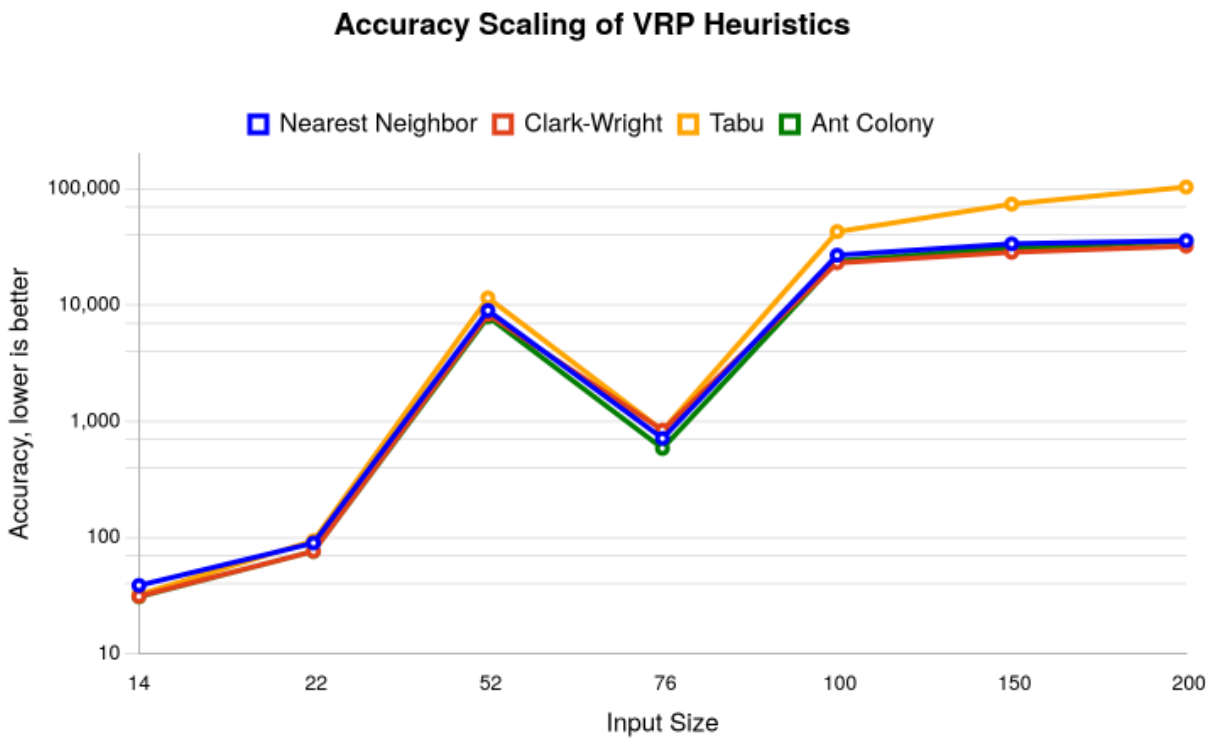
The objective of our study was to evaluate the runtime of Clarke-Wright, Nearest Neighbor, and Tabu Search, seeing how each scaled as runtime increased, determining at what point one algorithm becomes more efficient than the rest, and lastly we aimed to examine the tradeoff between runtime and solution accuracy. We expected simpler heuristics like Nearest Neighbor to scale efficiently with the tradeoff being less accurate or longer routes, while it was expected that more complex methods such as Tabu Search might produce solutions that were more accurate at the tradeoff of increased computational time. Ultimately, the aim was to identify how efficiency and solution quality balance against one another.

For the implementation of our algorithms we chose C++ due to the fact that it is a compiled language and our supposal that its nature of control over low-level memory would be helpful with our algorithms given that the programming required manipulation of data structures, such as vectors, and experimentation required many iterations throughout the process. Once we had the implementations for our algorithms, the experimentation consisted of tests using multiple input sizes for n (14, 22, 52, 76, 100, 150, and 200) using real datasets which consisted of nodes represented by Euclidean coordinates, where edge weights correspond to pairwise geometric distance and input size corresponds to the number of nodes or customers. These were taken from a GitHub repository which contained numerous sets from the ORSA Journal on Computing. For each input size, each algorithm was run for 20 trials per configuration to reduce variability in the results that might be caused by stochastic behavior, which lined up with our initial goal of testing all input sizes across 10-30 trials. Throughout each input size, the same dataset was used and the same number of trials, 20, remained for each algorithm. When measuring for accuracy, we took the total distance of the generated routes and compared qualitatively across each algorithm. After each trial had been run, we took the mean of runtimes for each algorithm to pinpoint the average runtime on each input size for further analysis.




Given the graph above, the empirical analysis shows that as input size n increases, the runtime of each algorithm grows, albeit at much different rates. Nearest Neighbor is the algorithm that exhibits the slowest growth, remaining extremely efficient across all tested input sizes which is expected given its expected behavior of $O(n^2)$ and low constant factors. Clarke-Wright shows moderate growth due to additional overhead from savings calculations which is expected given its expected behavior of $O(n^2 \log n)$. Tabu Search demonstrates growth that is significantly much more steep given its repeated neighborhood evaluations which is represented as $O(k*n)$. The most rapid increase in runtime comes from the Ant Colony Optimization which is a result of its dependence on multiple agents (ants) and iterations which results in a high computational cost. Overall, while each algorithm scales with input size, growth rates differ substantially with Ant Colony Optimization and Tabu Search scaling much worse that the constructive heuristics.

In terms of a crossover point in which one algorithm outperformed another, no such results were found with the range of n = 14 to n = 200. Nearest Neighbor consistently outperformed Clarke-Wright, which in turn consistently outperformed Tabu Search, which leaves Ant Colony Optimization as the slowest with every input size. Given no crossover point is observed, this would suggest the relative performance ordering is stable. There is, however, a key limitation within this experiment that could change this outcome: the limited range of input sizes. Given that the range of input sizes is relatively small and absence of large-scale instances where asymptotic behavior may become more pronounced, it is possible that a crossover point was not observed within the tested range. Further experimentation could be improved upon if repeated, and with the inclusion of larger datasets and incorporation of comparisons to best-known or optimal solutions, results could be gathered to suggest an overtaking from one algorithm to another. This could also prompt further research questions such as identifying the input size at which higher cost algorithms begin to justify runtime when compared to the quality of solutions produced by the algorithm in question. However, with the findings in this experiment, the ordering of scalability remains the same across each input size.




In the case of comparison between runtime and solution accuracy, the graph (shown above) bears unexpected results. Nearest Neighbor, Clarke-Wright, and Ant Colony Optimization perform similarly in terms of solution accuracy, clustering together throughout the range, with only slight deviations. Tabu Search initially follows this trend, but at larger input sizes (100-200), begins to diverge and produce noticeably worse solutions, even with its complexity. The current findings suggest that, with the current parameters, Tabu Search failed to outperform simpler heuristics and could possibly be counterproductive at scale. A drawback with this experiment is that, due to a lack of comparison to optimal solutions, accuracy can only be evaluated in a relative manner rather than as a true measurement. There is also the issue of having a restricted range of input sizes and specific number of iterations. If repeated, the experiment might benefit from the increase of such factors as range of input size, optimal solution comparison, and number of trials.

One of the main difficulties/roadblocks throughout the project came from the initial dataset that we wanted to use. Originally we only had a 15x15 distance matrix to work off of for our experimentation which we figured would be good. However, it became clear that experimentation would be very difficult on that one matrix alone. Fortunately, we found multiple datasets to test from which would make our research much more approachable. Unfortunately, a new problem arose when it became clear that our code needed a rework to accommodate the new format of our datasets. However, once that hurdle was overcome, we were able to get into the experimentation, slowly but sure. Overall, the project had its ups and downs, and while our process had a limited scope, it was still a good lesson learned on what to test for in the future and gave a clearer understanding into where faster heuristics and large scale implementations may be applicable.

## Sources

*Generative AI was used in the following:*

- Clarification of Time complexities for algorithms and clarification of the Vehicle Routing Problem for Checkpoint #1
- Design and Debugging of clarkWright.cpp and antColony.cpp
- Grammar check and clarity check when compared against final deliverables for 2400 Project Report

*Dataset Repository/journal repository is based off of*

https://github.com/mastqe/tsplib

[1] Reinelt, G. "TSPLIB--A Traveling Salesman Problem Library." ORSA Journal on Computing, Vol. 3, No. 4, pp. 376-384. Fall 1991.

.

## Table

| | **Adam Campbell** | **Tyler Gill** | **Tristan Worthington** |
|---|---|---|---|
| Brainstorming | 33.33% | 33.33% | 33.33% |
| Coding | 25% | 30% | 45% |
| Experiment Design | 33.3% | 33.3% | 33.3% |
| Visualization/Results | 35% | 20% | 45% |
| Communication | 30% | 40% | 30% |
| Report Writing | 60% | 20% | 20% |
| Presentation (creation) | 50% | 25% | 25% |
| Submission | 33.3% | 33.3% | 33.3% |